Nutri-MAS: a hybrid multi-agent system for personalized nutritional assistance

Acossi Giorgio Roberto - S5339095
acossigiorgio@gmail.com

July 2026

1 Details of the proposal

1.1 Complete specification of the proposal

Nutri-MAS is a conversational assistant that combines reactive agents, BDI agents, and LLM-based agents. The system collects the user's profile, calculates indicative caloric targets, builds a weekly plan, generates recipes, records the meals actually consumed, and rebalances the subsequent slots when the plan is modified. The main objective of the project is to develop a hybrid architecture capable of making deterministic systems collaborate with non-deterministic systems, more flexible and creative, exploiting the strengths of both approaches.

1.2 Type of proposal

Type of proposal is creative. The main objective is to experiment with how agents with different decision-making models can collaborate in the same system. *Nutri-MAS* combines three types of agents:

- **BDI agents**: they manage logic based on beliefs and rules, such as planning.
- **LLM agents**: they manage linguistic understanding and textual creativity.
- **Reactive agents**: they manage direct interaction with the user.

1.3 Difficulty level of the proposal

The proposed difficulty level depends greatly on the complexity of the implementation. The complexity of the project does not depend only on the number and quality of the functionalities implemented, but above all on the integration of

heterogeneous agents within a single operational flow. The system must translate requests expressed in natural language into structured commands and orchestrate the collaboration between BDI agents and LLM-based agents, characterized by different decision-making models and reasoning capabilities.

Some simplifications of the domain have been adopted to make the more complex aspects of meal planning manageable. These choices reduce the breadth of the problem, but they do not eliminate the main difficulty, represented by the end-to-end functional integration of the different components of the system. The difficulty of the project has been evaluated as *medium-hard*.

2 Introduction

Nutri-MAS is a hybrid multi-agent system for personalized nutritional assistance, meal planning, and recipe generation. This domain requires coordinating different needs: respecting the caloric, dietary, and temporal constraints of the individual user, proposing sufficiently varied recipes, and reacting to deviations from the plan. If, for example, the user consumes a meal different from the one expected, the system must record the event and redistribute the residual budget over the subsequent slots.

The objective of the project is to experiment with the collaboration between different decision-making models. Nutri-MAS assigns to BDI agents the logic based on beliefs and rules, such as planning and monitoring; to LLM agents it entrusts linguistic understanding and recipe generation; a reactive agent finally manages access to the system. Through the conversational interface the user can create their own profile, generate and consult a weekly plan, also record off-plan meals, and check the residual caloric budget.

2.1 High-level architecture

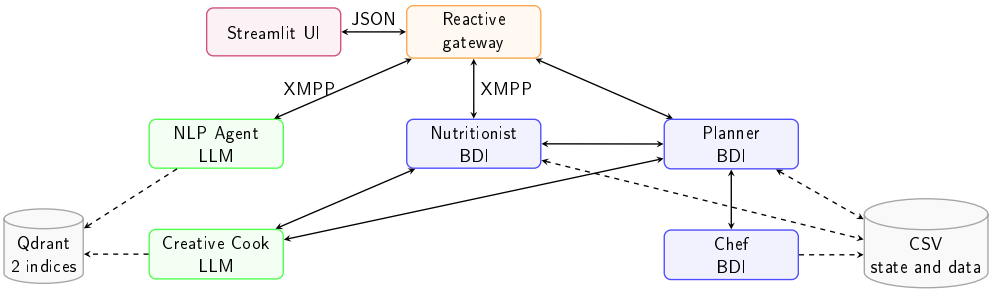

Figure 1: High-level architecture of Nutri-MAS

The architecture of Nutri-MAS is composed of a conversational interface (chat), a Gateway, and a set of specialized agents. The interface communicates with the

Gateway by means of JSON messages. The Gateway is a reactive agent that connects the frontend to the multi-agent system, validates the messages, and controls their routing.

The system includes three BDI agents, dedicated to state management and to the application of decision rules, and two LLM-based agents. The Nutritionist maintains the user profile, the nutritional targets, and the logs; the Planner builds the weekly plan, while the Chef selects the food templates compatible with the constraints. The NLP Agent interprets requests in natural language and produces structured commands; the Creative Cook instead uses the language model to generate concrete recipes starting from the templates and constraints defined by the BDI agents.

Communication between the agents takes place through XMPP, while the content of the application messages uses a representation compatible with the AgentSpeak syntax. State persistence is entrusted to CSV files that contain the user profile and the meal plans.

Qdrant supports the semantic search of the linguistic examples of the NLP Agent and of the ingredients used by the Creative Cook.

3 Technological stack

The system was developed using Python as the programming language and the SPADE, SPADE-BDI and SPADE-LLM libraries for agent management, Streamlit for the frontend, Qdrant for semantic search and LiteLLM as a proxy for access to Large Language Models.

3.1 Streamlit

The frontend is developed in Python and uses Streamlit to implement a *single-user* conversational interface. Communication between the UI and the Gateway takes place through two in-memory queues: one carries the user's messages toward the system, while the other transfers to the UI the responses and events produced by the agents.

The interface keeps message input available even during agent processing. The questions generated by the system preserve an explicit context, used by the NLP Agent to associate the user's response with the correct AgentSpeak command.

3.2 SPADE

SPADE (Smart Python Agent Development Environment) is a Python framework for the development of distributed multi-agent systems. The framework provides

the basic functionalities to define reactive agents, manage their life cycle and allow message exchange through the XMPP protocol. The behaviour of each agent is defined and organized through behaviours. To these functionalities are added the SPADE-BDI and SPADE-LLM extensions.

SPADE-BDI introduces a deliberative model based on beliefs, desires and intentions, in which the behaviour of agents is defined through AgentSpeak rules and plans.

SPADE-LLM makes it possible to integrate Large Language Models within agents, defining their behaviour through instructions and context and allowing them to use tools to access application functionalities and interact with the other components of the system.

Nutri-MAS combines these approaches by using reactive agents for communication and coordination, BDI agents for symbolic reasoning and LLM agents for natural language interpretation and creative activities.

3.3 XMPP

XMPP (Extensible Messaging and Presence Protocol) is an IETF standard for the near-real-time exchange of structured data through XML streams. The protocol defines mechanisms for addressing, authentication and message delivery, identifying each entity through an ID. In Nutri-MAS, each agent possesses a local ID used as identity and communication address. SPADE manages the connection to the XMPP server and the exchange of messages among agents, while the application content of the messages is represented through commands in AgentSpeak (ASL) syntax.

3.4 SPADE-BDI

The deliberative layer of Nutri-MAS is implemented through SPADE-BDI, an extension that allows the execution of AgentSpeak(L) programs within SPADE agents. The Nutritionist, Planner and Chef agents extend `BDIAgent` and load their respective AgentSpeak programs. In this model, *beliefs* represent the information that the agent considers true, *goals* describe the states that it intends to achieve and plans specify how to react to events. Each plan in fact defines a triggering event, a context that determines its applicability and a set of actions to execute.

Custom Python actions constitute the link between declarative reasoning and application functionalities, such as numerical calculations, access to CSV files, data serialization and the exchange of messages with non-BDI agents. This separation keeps decision-making policies explicit in the AgentSpeak programs, entrusting Python with computational and infrastructural operations.

3.5 SPADE-LLM

SPADE-LLM extends the SPADE agent model by introducing functionalities based on Large Language Models. An LLM agent is configured through a provider, a set of initial instructions (*system prompt*) and a series of tools, the tools.

The *tools* are implemented as Python functions and described through a schema that specifies their parameters and methods of use. The model can select the necessary tools, provide the corresponding arguments and use the obtained results to continue processing. A single request can therefore involve multiple interactions between the model and the tools, up to the production of the final result or the reaching of the configured limit.

In Nutri-MAS the LLM agents do not use a persistent conversational memory between successive requests. The context is built dynamically starting from the current message and from the information retrieved through the tools, which make it possible to access application functionalities and, when necessary, to request data from the other agents.

The NLP Agent uses this mechanism to identify candidate messages, validate their structure and translate natural language into AgentSpeak commands. The Creative Cook instead retrieves, through requests to the other agents and queries to the ingredient database, the data necessary to generate recipes or evaluate free meals. The LLMs therefore perform a role of local orchestration, coordinating the tools available within circumscribed workflows.

3.6 LiteLLM Proxy

LiteLLM provides a uniform abstraction layer for access to different LLM providers. In the project it is used as a proxy to expose both the APIs for LLM text generation and those for the production of embeddings. The model used was gpt-5.4 and text-embedding-3-large respectively for text generation and embedding.

3.7 Qdrant

Qdrant is a vector database designed to store embeddings and retrieve contents based on their semantic similarity. Compared to a search based on exact word matching, it makes it possible to identify conceptually close elements even when they are described with different expressions. Nutri-MAS uses it to support the semantic searches of the NLP Agent and the Creative Cook.

4 Data management

Nutri-MAS uses two complementary forms of data management: CSV files, used for persistence and for static datasets, and Qdrant collections, used for semantic search.

4.1 CSV data

To provide the agents with a static knowledge base and preserve the user's information across different executions of the application, Nutri-MAS uses a persistence system based on CSV files. This solution was preferred to a traditional database in order to reduce infrastructural complexity.

The CSV files are divided into two main categories: persistent files, read and updated during execution, and static datasets, used by the agents in read-only mode. Overall, they perform three functions:

- **Persistence**: preserves the user's state and history across successive executions.
- **Materialization**: stores the results produced by the planning process.
- **Static knowledge**: provides the agents with food templates, nutritional rules, and ingredient data.

During system startup, each agent loads or indexes the data under its responsibility. In the Nutritionist the operational and control state is represented by BDI beliefs; the Python code performs only adaptation, calculation, and persistence functions. Persistent data are updated on the files when the system is shut down.

File	Main agent	Description and use
`user.csv`	Nutritionist	Contains the user's profile, including anthropometric data, activity level, goals, allergens, diet type, and nutritional targets.
`weight_log.csv`	Nutritionist	Stores the weight history.
`meal_log.csv`	Nutritionist	Records planned and actually consumed meals, together with calories, macronutrients, and registration status.
`week_plan.csv`	Planner	Contains the complete weekly plan, with recipes, ingredients, instructions, and nutritional values.
`week_plan_template.csv`	Planner	Stores the abstract structure of the plan before the generation of the recipes.
`templates.csv`	Chef	Contains the candidate food templates, classified by slot, category, and nutritional values.
`nutrition_rules.csv`	Planner	Defines the variety and frequency constraints associated with food categories, diet types, and slots of the weekly plan.
`ingredients.csv`	Creative Cook	Contains the ingredients and their nutritional values per 100 grams. The dataset is semantically indexed to generate recipes and estimate freely described meals.

Table 1: Persistent data and reference datasets used by the agents

4.2 Qdrant collections

The vector collections are built at startup from the data and templates available in the project. During execution they are queried by the agents to retrieve the elements semantically closest to their requests.

Collection	Main agent	Description and use
`ingredients`	Creative Cook	Semantically indexes the names of the ingredients coming from the Australian Food Composition Database [2]. Each ingredient is associated with its calorie, protein, carbohydrate, and fat values. The Creative Cook uses the collection to identify the ingredients with which to develop the recipes and calculate their nutritional estimates.
`nlp_translations`	NLP Agent	Contains linguistic templates associated with the application messages available in the system. Through semantic search, the NLP Agent identifies the messages most suitable for the user's request and retrieves their structure from the registry. This information is then used to translate the request into a valid AgentSpeak command.

Table 2: Vector collections used by Nutri-MAS

The ingredient embeddings are generated through the provider configured in LiteLLM, while the NLP Agent's templates use an English embedding model configured directly in the index.

5 Design and implementation of the agents

The following table summarizes the main application functionalities associated with each agent.

Agent	Offered functionalities
Gateway	Integration between the UI and the MAS, including forwarding requests to the NLP Agent for translation, validating commands, and routing them
NLP	Translation of requests in natural language into valid application actions and conversion from AgentSpeak commands to responses in natural language
Nutritionist	Onboarding, target calculation, weight and meal logging, reminders, recaps and rebalancing of nutritional plans
Planner	Generation of the nutritional plan, control of variety and frequencies, plan consultation and management of daily meals
Chef	Selection of food templates compatible with diet, slot, calories, macronutrients and constraints
Creative Cook	Generation of concrete recipes and nutritional estimation of meals described freely

Table 3: Map between agents and application functionalities

5.1 SPADE behaviours and execution model

5.2 Gateway Agent

The Gateway connects the user interface to the multi-agent system and manages communication in both directions. It sends messages in natural language to the NLP, which translates them into AgentSpeak/Prolog commands, then validates and forwards those commands to the recipient agents. In the reverse path, it sends the technical responses of the agents back to the NLP so that they are converted into a form suitable for the UI. It does not make nutritional decisions, but deals only with the flow and validation of messages. Its operation is based on two cyclic behaviours. `LocalCommandBehaviour` retrieves from the UI queue the user's messages and sends them to the NLP for translation; `GatewayBehaviour` receives XMPP messages, forwards the translated commands to the domain agents and sends back to the NLP their responses, before publishing the result in the UI. Each XMPP send is executed by a one-shot `OutboundBehaviour`.

5.3 NLP Agent

The NLP Agent manages the translation between the natural language of the chat and the AgentSpeak/Prolog messages exchanged by the system. The two directions are implemented differently. The requests of the user are interpreted by an `LLMAgent`, while the responses of the BDI agents are mapped through a `CyclicBehaviour`.

The Gateway sends to the NLP Agent one of the following two formats:

Code 1: Input formats of the NLP Agent

```
username: <name>
message: <user text>

username: <name>
question: <question shown to the user>
answer: <user reply>
```

The first format represents an autonomous request, for example "Show me today's meal plan". The second is used when the user replies to a previous question. The Gateway includes explicitly both the displayed question and the new answer, making it possible to interpret the user's replies in the best way, which otherwise may be ambiguous.

A *registered message* represents an operation that the user can request through the chat. Its definition contains an identifier, a description, the recipient agent, the SPADE performative and the schema needed to build the AgentSpeak performative. All messages are saved in a JSON list, which constitutes the message registry.

Qdrant instead contains short linguistic templates used exclusively for retrieval. Each chunk associates a possible user formulation with the identifier of a message:

Code 2: Example of indexed linguistic chunk

```
{
    "nl_template": "I ate {actual_food} for {meal_type}.",
    "message": "nutritionist.log_meal"
}
```

Several chunks can point to the same identifier, making it possible to represent different formulations without duplicating the command definition. The search retrieves the semantically closest points, removes duplicate identifiers and returns a number of candidates chosen by the model, between one and ten. The content of the template therefore influences only the initial selection; it is never used as a command to execute.

For each retrieved identifier, the registry provides the complete definition.

Code 3: Abbreviated authoritative definition of nutritionist.log_meal

```
{
    "id": "nutritionist.get_daily_recap",
    "agent": "nutritionist",
    "message_performative": "achieve",
    "description": "Show today's calorie recap.",
    "signature": "get_daily_recap(Username)",
    "schema": {
        "functor": "get_daily_recap",
```

```
        "arity": 1,
        "type": "object",
        "properties": {
            "Username": {
                "type": "string",
                "source": "gateway"
            }
        },
        "required": ["Username"]
    }
}
```

The description helps the model distinguish semantically close operations, while signature and schema establish structure, order, types and allowed values. The model therefore must not reconstruct the command from the linguistic chunk, but only from the authoritative data of the registry.

The tools available for translation are:

- `search_message_templates`: searches in Qdrant for templates semantically similar to the request and returns the distinct identifiers of the candidate messages. The order of the results indicates only the degree of similarity.
- `get_message`: retrieves from the registry the authoritative definitions of the candidate messages, including description, recipient, performative, signature and argument schema.
- `submit_translation`: validates the ASL syntax, the arity, the absence of free variables and the consistency between command and registered route, then transmits the result to the Gateway.

The translation process follows the following steps:

- **Search**: the model performs semantic search to identify the candidate messages.
- **Selection**: it retrieves the corresponding definitions from the registry and selects the one consistent with the request.
- **Translation**: it fills the arguments with the data provided by the user and invokes `submit_translation`, which validates and transmits the result to the Gateway.

The system prompt describes the translation workflow and the ways of using the tools, without directly including the list of supported messages. The latter are defined in the registry and associated with the linguistic templates indexed in Qdrant. This separation makes the system extensible; in fact, it is possible to add new messages without modifying the system prompt, keeping the general logic of the NLP Agent unchanged.

5.4 Nutritionist Agent

The Nutritionist manages the nutritional state of the user and coordinates the daily cycle of the system. It maintains profile, objectives and logs, starts planning and reacts to events that require confirmations or modifications of the plan. The decision rules are expressed by means of BDI plans.

5.4.1 Beliefs used

The beliefs of the Nutritionist represent the user's profile and nutritional objectives, together with the history of meals and weight and the state of the operations in progress. The agent uses them to adapt targets, monitor consumption and decide when to update or rebalance the plan.

The BDI plans implement the following functionalities:

- **Onboarding**: `check_user_profile` verifies the presence of `user_profile_row`. If the profile exists, it derives the operational beliefs from it; otherwise it starts the questions on personal data, allergens, diet and culinary preferences, progressively storing the answers.
- **Profile update**: `update_weight` updates profile and `weight_log_entry`, then compares the new weight with the previous one. An absolute variation at least equal to 1% keeps the current goal, recalculates calories and macronutrients on the new weight and activates `refresh_week_plan`; smaller variations only update the state. Reaching the target has priority: it sets `maintain`, removes caloric deficit or surplus and regenerates the plan. `update_preferences` modifies the diet, while `update_culinary_preferences` replaces the free text of culinary preferences; these modifications too can regenerate the plan to avoid one that is no longer coherent.
- **Reminder**: a Python `CyclicBehaviour` publishes only the `clock_tick`. The BDI plan keeps `last_clock_tick` and generates a confirmation exclusively when the interval between two ticks crosses the slot time. The first tick, a date change or a backward jump of the clock establish a new baseline without recovering previous meals.
- **Meal tracking**: `confirm_meal` transfers recipe and planned values into the log row, marking it as `confirmed`. `log_meal` instead delegates to the Creative Cook the estimation of a different meal and saves the result as `modified`.
- **Rebalancing**: `request_rebalance_after` identifies the subsequent slots still modifiable, calculates the residual budget and requests new recipes from the Planner. The beliefs `pending_rebalances` and `pending_rebalance_origin` maintain the state of the workflow until completion of all responses.

- **Consultation**: `get_daily_recap`, `get_meal_logs` and `get_weight_logs` read the current beliefs and build the structured data returned to the Gateway.

The BDI plans and beliefs contain the state and establish when to execute the operations, while the Python actions implement only calculations, serialization and persistence. The temporal monitor does not maintain queues, flags or copies of the meal logs in Python. The Nutritionist collaborates with Planner for planning and rebalancing, with Chef for alternatives and with Creative Cook for the estimation of free meals.

The calculations are executed by Python actions called by the BDI plans. Starting from height and weight, the Nutritionist calculates the BMI and uses it to determine an initial objective of gain, maintenance or reduction of weight according to the reference thresholds. The daily caloric requirement is then estimated considering weight, height, age, sex and activity level and is corrected as a function of the objective.

The system compares each new weight with the last value saved in the profile. If the difference, in increase or in decrease, is at least equal to 1% of the previous weight, the Nutritionist keeps the active objective, recalculates the requirement on the new weight and requests the complete regeneration of the weekly plan. For variations below the threshold, instead, the new value is recorded without modifying the plan.

If the new weight reaches the established objective, the system switches to maintenance and regenerates the plan.

In case of deviation from the plan, the Nutritionist calculates the calories still available and redistributes them only among the subsequent meals, trying to rebalance the plan. This is a simplified model, based on coefficients and heuristics configured in the project.

5.5 Planner Agent

The Planner transforms the user's nutritional requirement into a weekly plan. It coordinates the construction of the plan, keeps the progress state and waits for the responses of Chef and Creative Cook before proceeding. The planning and control rules are expressed by means of BDI plans.

5.5.1 Beliefs used

The beliefs of the Planner represent the received targets, the plan already built, the constraints of variety and the progress state of the planning. The agent consults and updates them to coordinate the choice of meals, avoid repetitions and proceed correctly from one slot to the next.

The BDI plans implement the following functionalities:

- **Complete construction**: `build_week_plan` saves user, diet and targets in the belief `planning_context`, initializes `next_step` to Monday/breakfast and prevents the start of two concurrent plannings.
- **Nutritional distribution**: for each slot the plan calculates calories and macros and stores them in `slot_macro_target`. `slot_preferred_category` consults the current frequencies to favor the categories still below the weekly minimum.
- **Variety and local backtracking**: `used_dish` records the dishes already chosen, while `category_count` maintains the category frequencies. If the Chef's response violates one of these constraints, the Planner repeats the request locally by adding the element to the exclusions.
- **Recipe composition**: after template selection, `pending_slot` and `pending_recipe_slot` keep the context of the request sent to the Creative Cook. The belief is removed only when `recipe` arrives; the result is saved and `advance_plan` updates `next_step`.
- **Final validation**: at the end of the slots, `send_final_plan` compares `category_count` with the configured limits. A failure interrupts the workflow, avoiding publishing as valid an incomplete or non-compliant plan.
- **Local replanning**: `rebalance_slot` stores the slot concerned and applies the new target received from the Nutritionist, rebuilding only that meal without restarting the entire week.
- **Consultation**: `get_plan`, `query_plan` and `get_plan_day_context` read the beliefs `planned_dish_row` and `planned_recipe_row` to return the complete plan, single slot or daily context.

The BDI plans establish the order of decisions, while the Python actions calculate the targets per slot, determine the next step and serialize the result. The Planner receives profile and objectives from the Nutritionist, uses the Chef to select the templates and the Creative Cook to generate the recipes.

5.6 Chef Agent

The Chef selects food templates compatible with the constraints of the plan. It does not directly generate the recipes, but narrows the space of alternatives considering diet, slot, category, calories, macronutrients and nutritional constraints. The selection rules are expressed by means of BDI plans and rules.

5.6.1 Beliefs used

The beliefs of the Chef represent the available food templates and the criteria with which to evaluate their compatibility with respect to the current request. The agent uses them to filter the alternatives according to dietary and nutritional constraints and return to the requester an applicable candidate.

The BDI plans implement the following functionalities:

- **Compatibility with diet and slot**: the beliefs `template_for_plan` represent the available templates. The rules `diet_category_allowed` and `slot_allowed` eliminate those incompatible with the user's diet or with the requested slot.
- **Nutritional tolerance**: `strict_template_candidate` combines calories and macronutrients of the template with the received target. The beliefs `tollerance` delimit the acceptable intervals before a candidate can be selected.
- **Guided category**: when the Planner still has to satisfy a minimum frequency, `guided_strict_template_candidate` adds the required category to the selection constraints.
- **Variety**: the names and categories received as exclusions are discarded during candidate construction. `.random_candidate` introduces variability only after the application of all the rules.
- **Controlled fallback**: if no strict candidates exist, the plan searches for a compatible template under 500 kcal. If this set too is empty, it returns an explicit failure to the requester.
- **Meal alternative**: `choose_runtime_template` consults the template beliefs and selects a proposal different from the planned meal, maintaining compatibility with the slot.

The BDI plans establish which candidates are applicable, while the Python actions load the templates, verify the nutritional tolerances and select one element among the valid ones. The Chef receives the targets from the Planner and the alternative requests from the Nutritionist, returning a structured template or an explicit failure.

5.7 Creative Cook Agent

The Creative Cook deals with managing two distinct operations. During planning it transforms an abstract template into a concrete recipe with name, ingredients, grammages, instructions and macronutrients. During tracking, instead, it interprets a meal described freely and estimates its composition and nutritional values, also producing the template to use for the subsequent rebalancing.

5.7.1 Data used

The Creative Cook does not maintain a BDI belief base: it accesses the user's nutritional context, the constraints and targets of the request and the nutritional data of the ingredients. It uses this information to generate compatible recipes or estimate the composition of a free meal.

The Creative Cook receives one of the following two AgentSpeak messages:

Code 4: Input formats of the Creative Cook Agent

```
prepare_plan_slot(Username, Day, MealType, Template, TargetCalories,
    TargetProtein, TargetCarbs, TargetFat)
evaluate_free_meal(Username, Date, MealType, MealName, UserCalories)
```

The first format is sent by the Planner and contains the selected template together with the targets of the slot. The second comes from the Nutritionist when the user declares having consumed a meal different from the planned one; `UserCalories` can contain a value declared by the user. The signature of the message determines the workflow and the allowed terminal tool, preventing recipe generation and consumption evaluation from being combined.

The available tools are:

- `query_user_nutrition_context`: requests from the Nutritionist, in read-only mode, diet, allergens, culinary preferences, objective, daily calories and meal distribution, the response is handled through a dedicated behaviour.
- `query_ingredient_db`: performs a semantic search in the Qdrant database of ingredients with nutritional values per 100 g.
- `submit_prepared_recipe`: builds the message `recipe` with ingredients and grammages, instructions, calories and macros, then sends it to the Planner through the performative `prepare_plan_slot`.
- `submit_evaluated_free_meal`: builds the message `free_meal_evaluated` with estimated calories, macros and template, then sends it to the Nutritionist through the performative `evaluate_free_meal`.

The process of preparing a recipe follows the following steps:

- **Interpretation**: the model interprets the template and retrieves the nutritional context, checking allergens and diet first and then culinary preferences.
- **Search**: it searches semantically for the necessary ingredients, covering the main groups of the recipe, and uses only ingredients returned by the database.
- **Calculation**: it assigns grammages, estimates calories and macros from the values per 100 g and performs the estimation of portions with respect to the

targets.

- **Sending**: it invokes `submit_prepared_recipe` to send the recipe to the Planner.

The process of evaluating a free meal follows the following steps:

- **Search**: the model searches in the database for the full name of the meal, verifying whether there is a correspondence with an already prepared food.
- **Decomposition**: if the result is not sufficient, it decomposes the dish into its main ingredients and searches for them separately.
- **Estimation**: it calculates portions, calories and macronutrients; if the user indicated the calories, it adapts the macros to the declared total.
- **Classification**: it chooses the template that best represents the consumed meal.
- **Sending**: it invokes `submit_evaluated_free_meal` to send the evaluation to the Nutritionist.

The system prompt encodes both the routing and the hierarchy of constraints. Allergens are a strong constraint, the diet must always be respected and culinary preferences are applied only when compatible with safety, template, available ingredients and targets. If no specific preferences are present, the model favors traditional Italian cuisine. The text of the preferences is treated exclusively as recipe data and cannot modify tools, workflow or output format.

Allergens are free text and are treated by the prompt as a strong constraint: in case of doubt the model must avoid the ingredient and choose a safer alternative. The instructions of the recipe remain brief and do not repeat doses or the list of ingredients, already transmitted separately. These constraints entrusted to the model are useful precautions, but they are not equivalent to a deterministic clinical validation. The limit of eight interactions also reduces the risk of search cycles.

5.8 Communication between BDI agents and non-BDI agents

All agents communicate by means of messages transported over XMPP. The difference therefore does not concern the communication protocol, but the way in which each type of agent interprets the received messages. The LLM agents process requests through the SPADE-LLM behaviours, which coordinate the local context of the LLM with the tool calls.

When a normal SPADE agent, such as the Gateway or the Creative Cook, sends a message to a BDI agent, `TellBridge` and `AchieveBridge` adapt the application protocol to AgentSpeak events. Both analyze the ASL body of the message:

`TellBridge` translates a `tell` performative into the addition of a belief, while `AchieveBridge` translates an `achieve` performative into an achievement goal.

Between two BDI agents, on the other hand, no bridge is necessary. The AgentSpeak action `.send` provided by SPADE-BDI builds an XMPP message in the native format of the extension and the recipient's `BDIBehaviour` converts it directly into the corresponding event. The bridges therefore remain confined to the connection between BDI and non-BDI agents.

6 Workflow

6.1 Onboarding and first planning

At startup the Nutritionist checks whether the user's profile exists. Otherwise it collects the data needed, repeating the request for each missing field; then it calculates the goal and the daily requirement and starts the generation of the weekly plan.

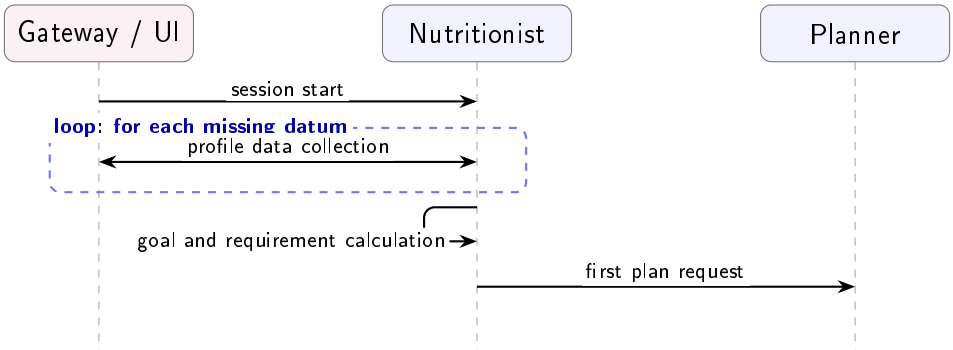

Figure 2: UML diagram of the onboarding workflow

6.2 Construction of the weekly plan

The Planner coordinates the construction of the weekly plan. For each food slot it determines the nutritional target to be satisfied; the Chef then selects a compatible template, while the Cook materializes the selected template into a concrete recipe. The generation was deliberately divided into two phases. In the first phase, the BDI agents build a structured and deterministic representation of the plan using food templates. These templates represent the basic structure of meals and specify the structure of the recipe, the categories of allowed ingredients and the indicative values of calories and macronutrients. The Chef selects, for each slot, a template compatible with the nutritional target and with the available constraints. In the second phase, the selected template is entrusted to the Cook, which transforms the template into a complete recipe, defining its ingredients,

quantities and preparation in compliance with the preferences and constraints of the user. The completed slots are transferred to the Nutritionist as beliefs. Temporal monitoring does not perform recoveries at startup: a slot can produce a request only if the system, already running, crosses its configured time.

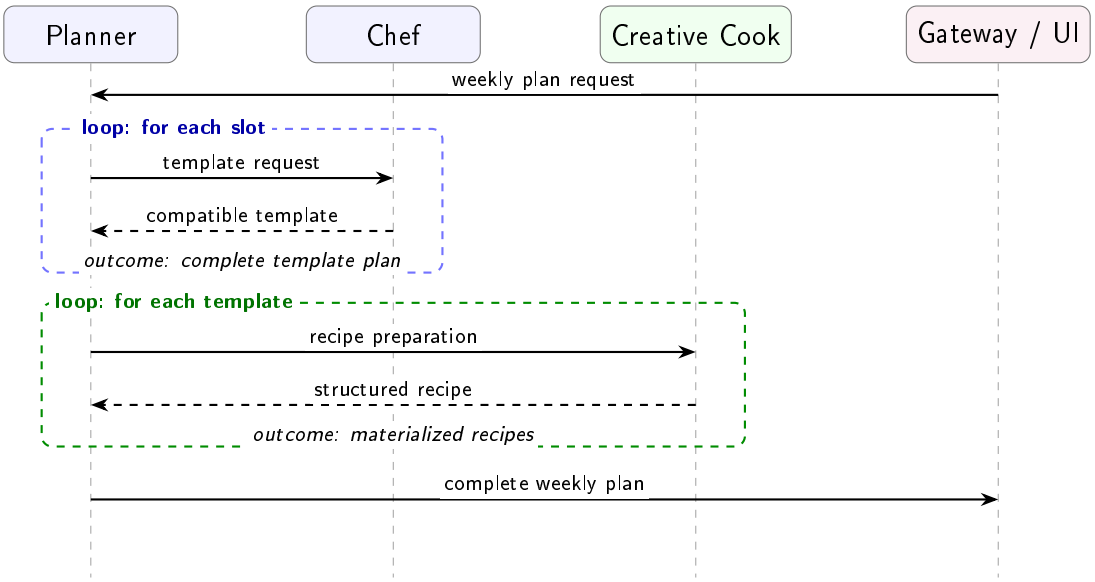

Figure 3: UML diagram of the construction of the weekly plan

6.3 Weight update and plan regeneration

When the user communicates a new weight, the Nutritionist updates the profile and the history and compares the measurement with the weight previously stored in the profile. If the absolute variation reaches 1%, the agent recalculates the requirement on the new weight while maintaining the current goal and regenerates the entire weekly plan; below the threshold it records the value without modifying the plan. This logic also operates between entries made on the same day.

The verification of the goal precedes the percentage heuristic. For a weight-loss goal the target is reached with a weight less than or equal to the reference; for a gain goal with a weight greater than or equal to it. In that case the goal switches to `maintain`, the requirement is recalculated on the reached weight without deficit or surplus and the week is regenerated in maintenance mode.

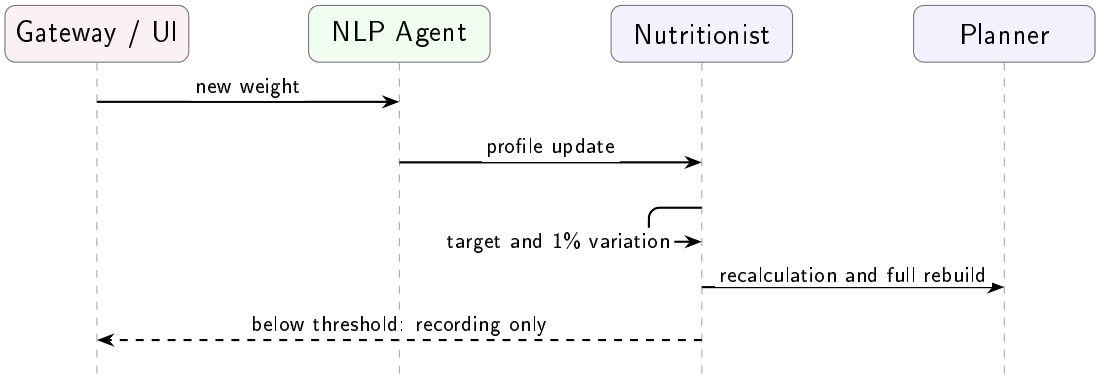

Figure 4: UML diagram of the weight update

6.4 Proactive monitoring, confirmation and different meal

The Nutritionist periodically receives a tick from the Python monitor and compares in the BDI beliefs the current time with that of the previous tick. It requests confirmation only for a slot whose time is crossed while the system is running. The first tick establishes the baseline: if the application is started at 14:00, breakfast, morning snack and lunch are not recovered nor asked later. Therefore there is neither a queue of expired meals nor a duplicated state machine in Python. The check is repeated at every tick for the entire duration of the session. The user can confirm what was planned or indicate that they consumed a different meal. In the latter case, the Cook estimates its nutritional values and the Nutritionist possibly rebalances the subsequent meals of the same day to compensate for any deviation from the plan.

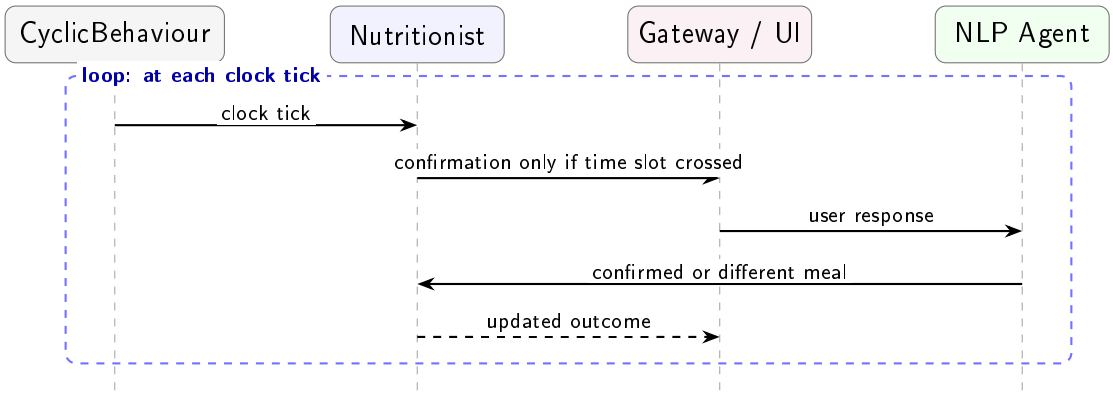

Figure 5: UML diagram of meal monitoring

6.5 Replacement and rebalancing after a deviation

When a different meal is recorded, the Nutritionist compares the actual consumption with the daily requirement and recalculates only the future meals of the same day. The Planner, the Chef and the Cook then look for new recipes that compensate for the deviation while maintaining the diet and constraints of the plan. The sequence is repeated for each future slot still modifiable. Compensation is limited by the availability of templates and realistic recipes, the system does not force excessively reduced meals to recover the deviation.

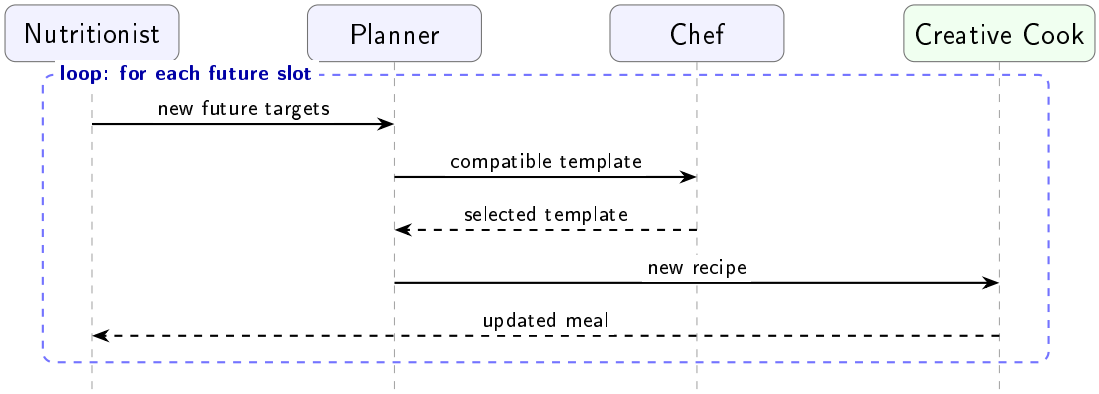

Figure 6: UML diagram of rebalancing

6.6 Plan consultation, recap and history

The user can request the current plan, daily recap and meal or weight history. The information is retrieved and presented in structured format.

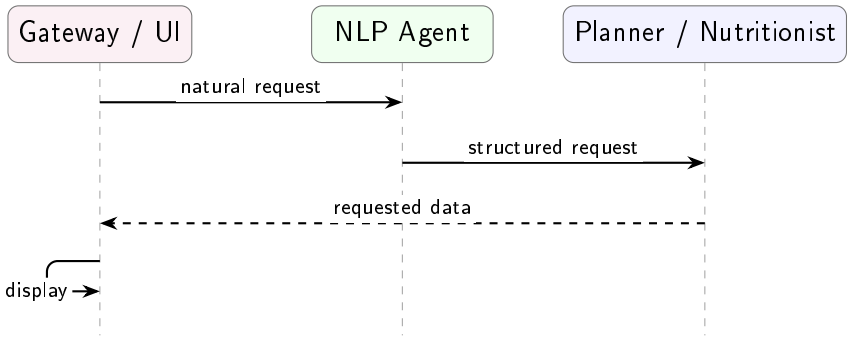

Figure 7: UML diagram of data consultation

7 Demo

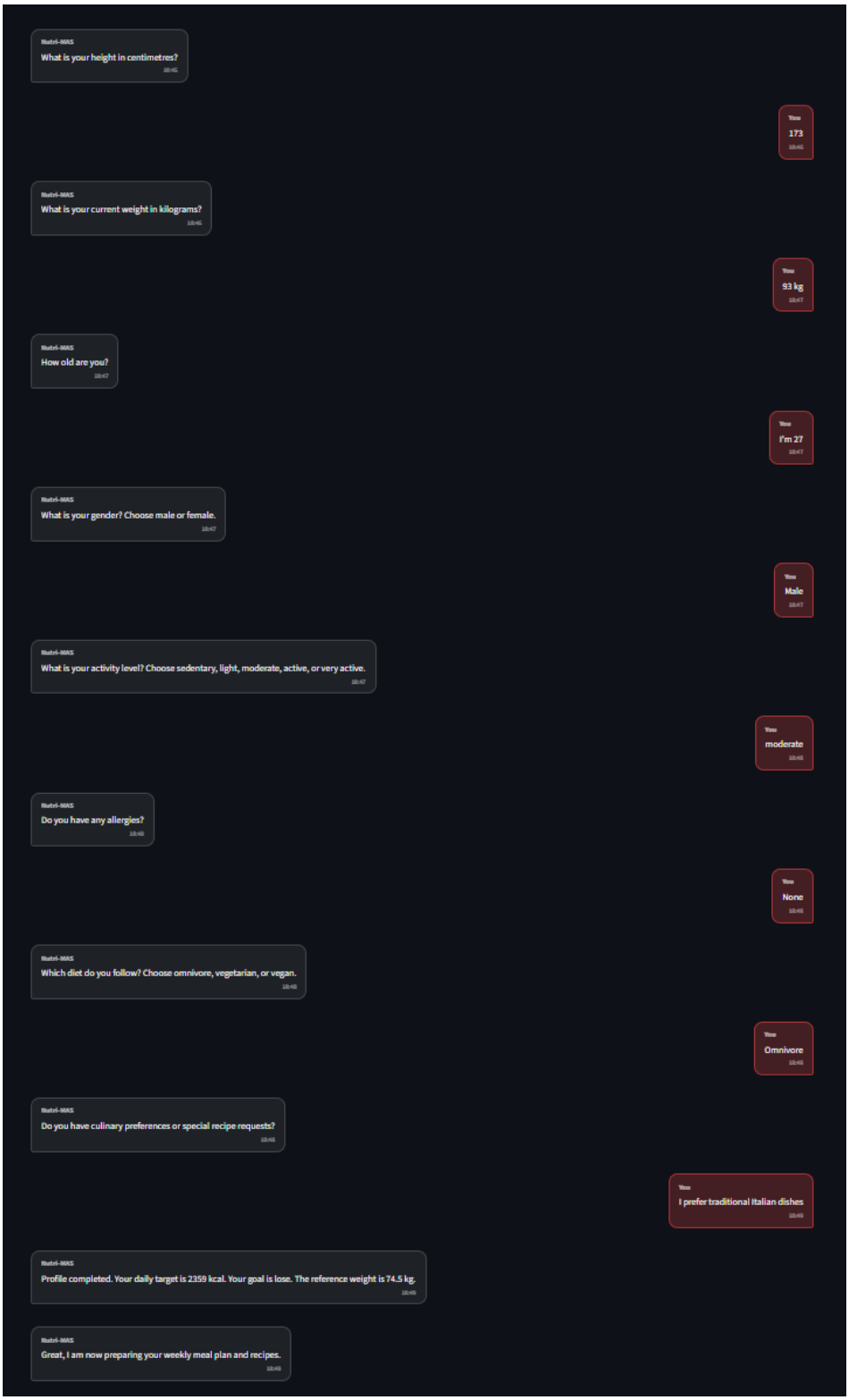

Figure 8: Onboarding and initial target calculation

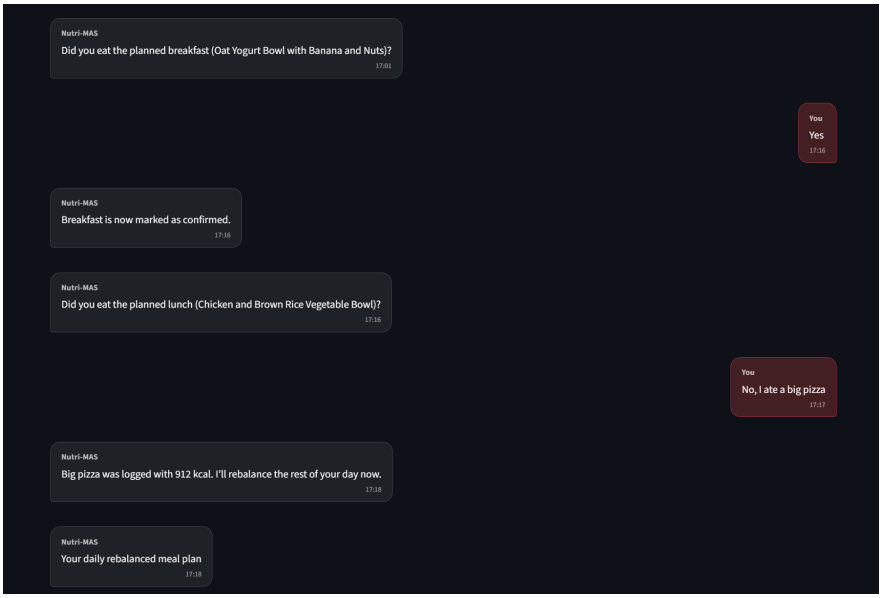

Figure 9: Monitoring and daily rebalancing

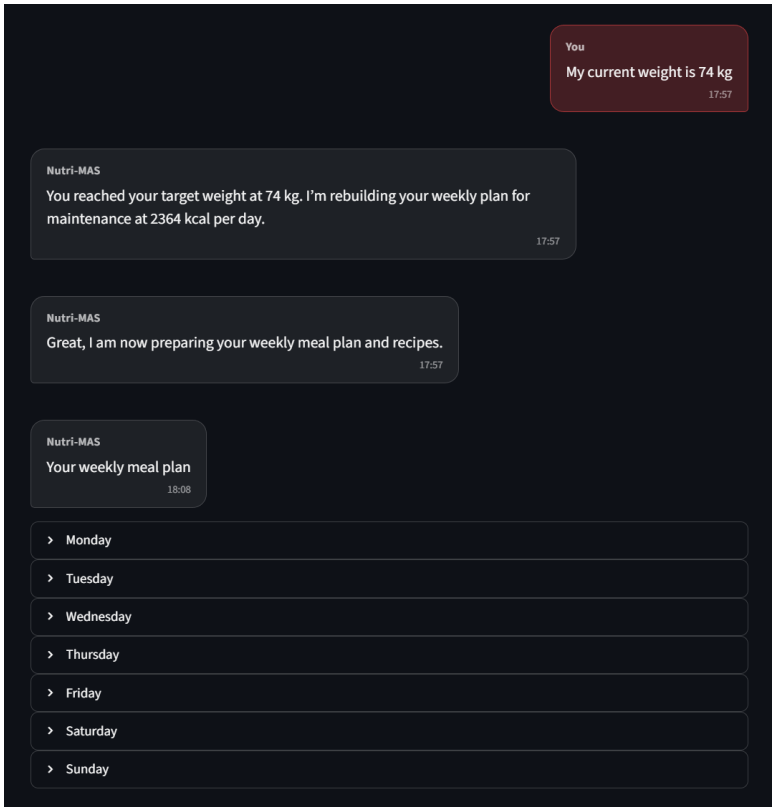

Figure 10: Transition to maintenance after reaching the target

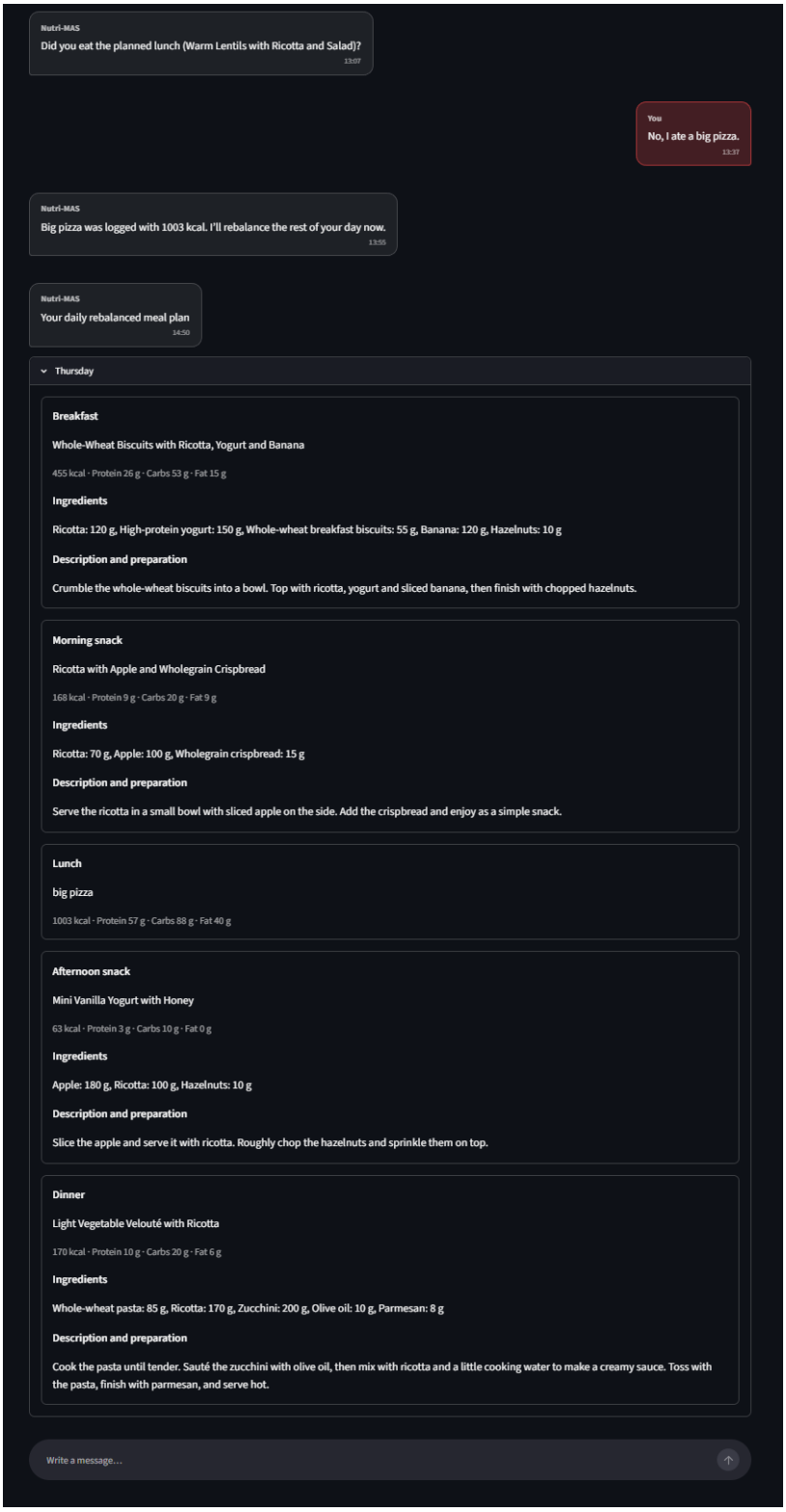

Figure 11: Rebalancing of the weekly plan after a deviation

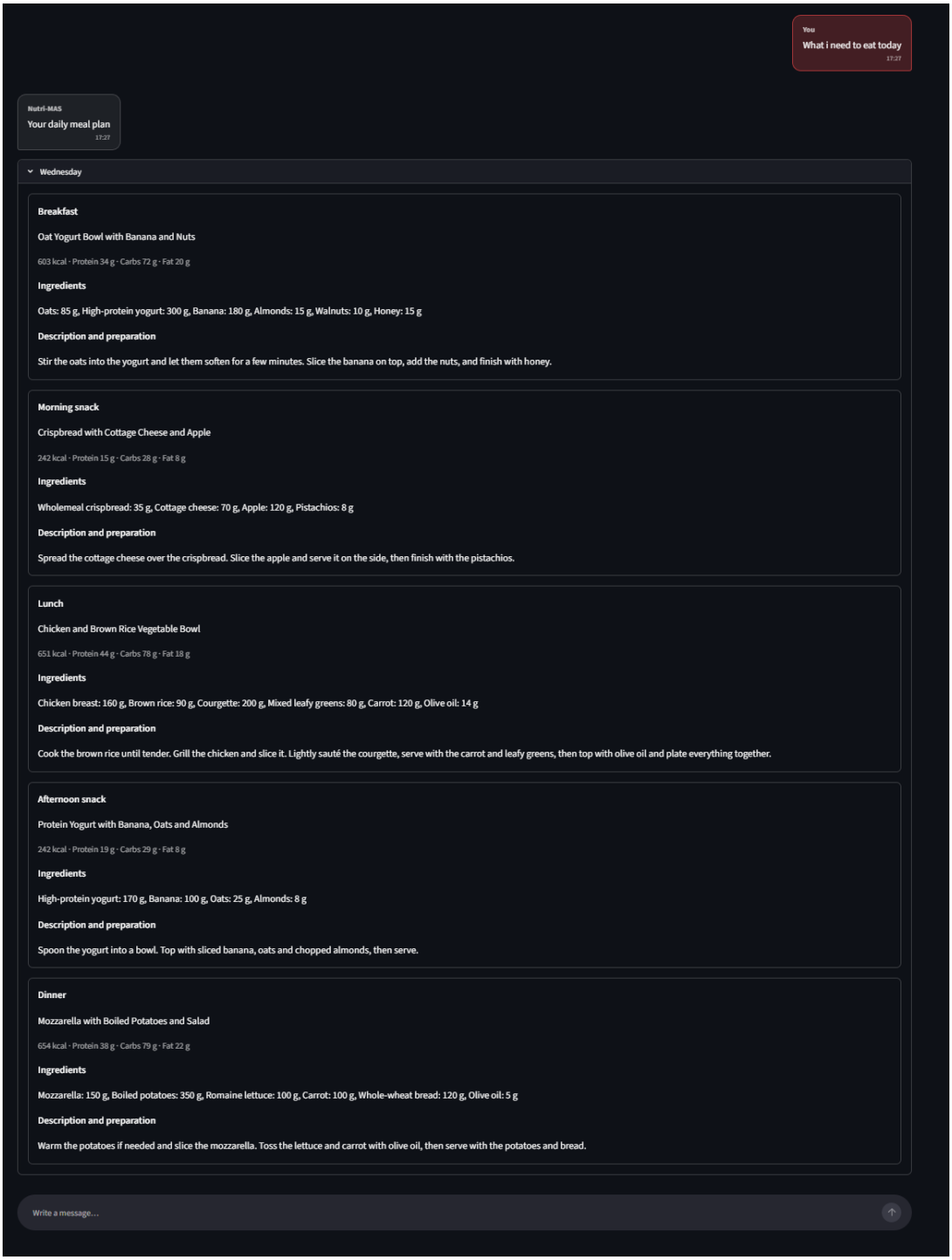

Figure 12: Consultation of the daily meal plan

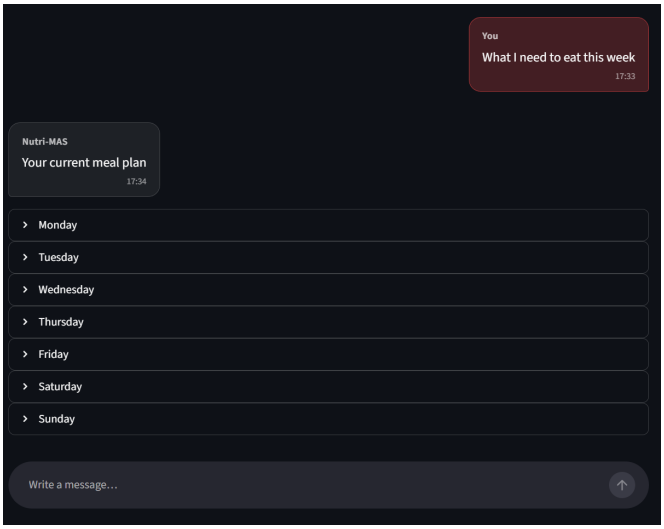

Figure 13: Consultation of the weekly meal plan

8 Limitations and open issues

- **Translation from natural language**: the conversion of the user's requests into structured constraints is based on a finite set of manually defined templates and rules. The system can therefore prove fragile with respect to synonyms, unforeseen formulations, implicit constraints, or ambiguous requests.
- **Limited coverage of meal templates**: the templates used to build meals constitute a selected and manually encoded subset. Consequently, the variety of the generated plans depends on the coverage of the templates and does not include all the possible combinations of ingredients and recipes.
- **Simplification of the nutritional model**: the calculation of calories and macronutrients uses average values coming from the dataset and depends on the matching of ingredients and on the portions adopted. Furthermore, the generation of recipes through LLM can introduce ingredients or quantities not fully consistent with the selected template.
- **Heuristic definition of targets**: the nutritional targets are calculated by means of predefined formulas and coefficients applied uniformly to the different profiles. This approach simplifies the estimation, but does not model all the variables that influence the actual individual requirement.
- **Simplified weight trend**: the continuous recalculation compares only two consecutive weigh-ins and uses an absolute threshold of 1%, without moving averages, minimum frequency of measurements, or correction for daily fluctuations. This choice makes the behavior deterministic and demonstrable in real time, but is not equivalent to a clinical nutritional monitoring protocol.
- **Non-retrospective reminders**: to keep the BDI behavior simple and observable, notifications require that the system be active while the time slot is being crossed. Meals prior to startup or to a clock jump are not reconstructed; the day's meal log can therefore be intentionally incomplete.
- **Local caloric rebalancing**: the rebalancing corrects the caloric deviation through a simplified modification of quantities. The procedure does not jointly optimize calories, macronutrients, food frequencies, variety, and compliance with constraints over the entire week.
- **Simplified management of allergens and dietary constraints**: allergens and preferences expressed in free text are translated into instructions for the Cook agent, but a structured representation of the relationships among ingredients, derivatives, food categories, and possible substitutions is not available. This limits the systematic verification of consistency between the declared constraints and the selected ingredients.

9 How to run the project

From the repository root, it is advisable to create and activate a Python virtual environment, then install the dependencies listed in `requirements.txt`:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Before startup, the two local configuration files must be prepared. The repository includes example versions without credentials, which can be copied with the following commands:

```
Copy-Item config\.env.example config\.env
Copy-Item config\litellm_config.example.yaml `
    config\litellm_config.yaml
```

To start the entire system, the PowerShell script included in the repository is recommended:

```
.\scripts\run.ps1
```

References

- M. D. Mifflin et al., "A New Predictive Equation for Resting Energy Expenditure in Healthy Individuals", 1990. Article from which the equation for estimating resting energy expenditure is taken. https://doi.org/10.1093/ajcn/51.2.241
- Food Standards Australia New Zealand, "Australian Food Composition Database – Release 3", 2025. Dataset from which the nutritional values of the ingredients come. https://www.foodstandards.gov.au/science-data/food-nutrient-databases/afcd/data-files
- Centers for Disease Control and Prevention, "Adult BMI Categories". Reference for the BMI thresholds used in determining the initial goal. https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html